

7. Gyűjtemények (Collections) C#-ban

Mi az a gyűjtemény?

A gyűjtemények (collections) olyan adatszerkezetek, amelyek **több elemet** tárolnak, és **rugalmasabb** módot biztosítanak az adatok kezelésére, mint a tömbök. A gyűjtemények a `System.Collections.Generic` névtérben találhatók.

Főbb különbségek a tömbhöz képest:

- **Dinamikus méret:** Automatikusan növekszik/csökken
- **Típusbiztos:** Generic típusok használata
- **Gazdag API:** Sok beépített metódus

```
using System.Collections.Generic; // Fontos!
```

List - Dinamikus lista

A `List<T>` a leggyakrabban használt gyűjtemény típus. Olyan, mint egy tömb, de a mérete automatikusan változik.

List létrehozása

```
// Üres lista létrehozása
List<int> szamok = new List<int>();

// Lista inicializálása értékekkel
List<int> szamok2 = new List<int> { 1, 2, 3, 4, 5 };

// Rövidebb szintaxis (C# 3.0+)
List<string> nevek = new List<string> { "Anna", "Béla", "Cecil" };

// Új szintaxis (C# 9.0+)
List<double> atlagok = new() { 4.5, 3.8, 5.0 };

// Különböző típusok
List<int> egesz = new List<int>();
List<string> szoveg = new List<string>();
List<double> lebego = new List<double>();
List<bool> logikai = new List<bool>();

// Kezdő kapacitással (optimalizálás)
List<int> nagyLista = new List<int>(1000); // 1000 elem helye előre lefoglalva
```

Elemek hozzáadása

```
List<string> gyumolcsok = new List<string>();

// Add - egy elem hozzáadása a végére
gyumolcsok.Add("alma");
gyumolcsok.Add("körte");
gyumolcsok.Add("barack");
// gyumolcsok: ["alma", "körte", "barack"]

// AddRange - több elem hozzáadása
string[] ujGyumolcsok = { "szilva", "cseresznye" };
gyumolcsok.AddRange(ujGyumolcsok);
// gyumolcsok: ["alma", "körte", "barack", "szilva", "cseresznye"]

// Insert - beszúrás adott pozícióra
gyumolcsok.Insert(1, "banán"); // 1. indexre szúrja be
// gyumolcsok: ["alma", "banán", "körte", "barack", "szilva", "cseresznye"]

// InsertRange - több elem beszúrása
List<string> ujak = new List<string> { "narancs", "citrom" };
gyumolcsok.InsertRange(2, ujak);

// Példa számokkal
List<int> szamok = new List<int>();
szamok.Add(10);
szamok.Add(20);
szamok.Add(30);
Console.WriteLine(szamok.Count); // 3
```

Elemek elérése

```
List<string> nevek = new List<string> { "Anna", "Béla", "Cecil", "Dóra" };

// Indexelés (mint a tömbök)
string elso = nevek[0]; // "Anna"
string masodik = nevek[1]; // "Béla"
string utolso = nevek[3]; // "Dóra"

// Utolsó elem
string utolso2 = nevek[nevek.Count - 1]; // "Dóra"

// Elem módosítása
nevek[0] = "Annamária";
// nevek: ["Annamária", "Béla", "Cecil", "Dóra"]

// Index túllépés - HIBA!
// string hiba = nevek[10]; // ArgumentOutOfRangeException!

// Biztonságos hozzáférés
int index = 2;
if (index >= 0 && index < nevek.Count)
```

```
{  
    Console.WriteLine(nevek[index]);  
}
```

Count és Capacity

```
List<int> szamok = new List<int> { 1, 2, 3, 4, 5 };  
  
// Count - elemek tényleges száma  
int elemSzam = szamok.Count; // 5  
  
// Capacity - lefoglalt kapacitás (méret nélkül)  
int kapacitas = szamok.Capacity; // általában nagyobb mint Count  
  
Console.WriteLine($"Elemek: {elemSzam}, Kapacitás: {kapacitas}");  
  
// Példa a különbségre  
List<int> lista = new List<int>();  
Console.WriteLine($"Count: {lista.Count}, Capacity: {lista.Capacity}"); // 0, 0  
  
lista.Add(1);  
Console.WriteLine($"Count: {lista.Count}, Capacity: {lista.Capacity}"); // 1, 4  
  
lista.Add(2);  
lista.Add(3);  
lista.Add(4);  
lista.Add(5); // Capacity automatikusan növekszik  
Console.WriteLine($"Count: {lista.Count}, Capacity: {lista.Capacity}"); // 5, 8
```

Elemek eltávolítása

```
List<string> nevek = new List<string> { "Anna", "Béla", "Cecil", "Béla", "Dóra" };  
  
// Remove - első előfordulás eltávolítása (érték alapján)  
bool sikeres = nevek.Remove("Béla"); // true, eltávolítja az első "Béla"-t  
// nevek: ["Anna", "Cecil", "Béla", "Dóra"]  
  
bool sikertelen = nevek.Remove("Erik"); // false, nincs ilyen elem  
  
// RemoveAt - eltávolítás index alapján  
nevek.RemoveAt(0); // Eltávolítja az "Anna"-t  
// nevek: ["Cecil", "Béla", "Dóra"]  
  
// RemoveRange - több elem eltávolítása  
List<int> szamok = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8 };  
szamok.RemoveRange(2, 3); // 2. indextől 3 elemet töröl  
// szamok: [1, 2, 6, 7, 8]  
  
// RemoveAll - feltétel alapján törlés
```

```
List<int> szamok2 = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
szamok2.RemoveAll(x => x % 2 == 0); // Páros számok törlése
// szamok2: [1, 3, 5, 7, 9]

// Clear - minden elem törlése
nevek.Clear();
Console.WriteLine(nevek.Count); // 0
```

Keresés és ellenőrzés

```
List<string> gyumolcsok = new List<string> { "alma", "körte", "barack", "szilva" };

// Contains - tartalmaz-e elemet
bool vanAlma = gyumolcsok.Contains("alma"); // true
bool vanNarancs = gyumolcsok.Contains("narancs"); // false

if (gyumolcsok.Contains("alma"))
{
    Console.WriteLine("Van alma!");
}

// IndexOf - elem első előfordulásának indexe
int index1 = gyumolcsok.IndexOf("körte"); // 1
int index2 = gyumolcsok.IndexOf("narancs"); // -1 (nincs benne)

// LastIndexOf - utolsó előfordulás
List<int> szamok = new List<int> { 1, 2, 3, 2, 5 };
int elso = szamok.IndexOf(2); // 1
int utolso = szamok.LastIndexOf(2); // 3

// Exists - van-e olyan elem, ami megfelel a feltételnek
bool vanNagyobb5 = szamok.Exists(x => x > 5); // false
bool vanParos = szamok.Exists(x => x % 2 == 0); // true

// Find - első elem, ami megfelel
int elsoParos = szamok.Find(x => x % 2 == 0); // 2

// FindAll - minden elem, ami megfelel
List<int> parosak = szamok.FindAll(x => x % 2 == 0);
// parosak: [2, 2]

// FindIndex - első illeszkedő elem indexe
int parosIndex = szamok.FindIndex(x => x % 2 == 0); // 1
```

Rendezés és megfordítás

```
List<int> szamok = new List<int> { 50, 10, 30, 20, 40 };
```

```
// Sort - rendezés növekvő sorrendbe
szamok.Sort();
// szamok: [10, 20, 30, 40, 50]

foreach (int szam in szamok)
{
    Console.WriteLine(szam);
}

// Reverse - megfordítás
szamok.Reverse();
// szamok: [50, 40, 30, 20, 10]

// String rendezés
List<string> nevek = new List<string> { "Cecil", "Anna", "Béla" };
nevek.Sort();
// nevek: ["Anna", "Béla", "Cecil"]

// Egyéni rendezés (csökkenő sorrend)
List<int> szamok2 = new List<int> { 5, 2, 8, 1, 9 };
szamok2.Sort();
szamok2.Reverse();
// szamok2: [9, 8, 5, 2, 1]

// Lambda kifejezéssel (később tanulod részletesen)
List<int> szamok3 = new List<int> { 5, 2, 8, 1, 9 };
szamok3.Sort((a, b) => b.CompareTo(a)); // Csökkenő
```

Lista bejárása

```
List<string> nevek = new List<string> { "Anna", "Béla", "Cecil" };

// for ciklus
for (int i = 0; i < nevek.Count; i++)
{
    Console.WriteLine($"{i}: {nevek[i]}");
}

// foreach ciklus (leggyakoribb)
foreach (string nev in nevek)
{
    Console.WriteLine(nev);
}

// ForEach metódus (funkcionális stílus)
nevek.ForEach(nev => Console.WriteLine(nev));

// while ciklus
int index = 0;
while (index < nevek.Count)
{
```

```
        Console.WriteLine(nevek[index]);  
        index++;  
    }
```

Egyéb hasznos metódusok

```
List<int> szamok = new List<int> { 1, 2, 3, 4, 5 };  
  
// ToArray - lista → tömb konverzió  
int[] tomb = szamok.ToArray();  
  
// GetRange - részlista másolása  
List<int> reszLista = szamok.GetRange(1, 3); // [2, 3, 4]  
  
// TrueForAll - minden elem megfelel-e  
bool mindenPozitiv = szamok.TrueForAll(x => x > 0); // true  
  
// ConvertAll - átalakítás  
List<string> szovegek = szamok.ConvertAll(x => x.ToString());  
// szovegek: ["1", "2", "3", "4", "5"]  
  
// Másolás  
List<int> masolat = new List<int>(szamok); // Új lista ugyanazokkal az elemekkel
```

Gyakorlati List példák

```
// Bevásárló lista  
List<string> bevasarloLista = new List<string>();  
bevasarloLista.Add("Kenyér");  
bevasarloLista.Add("Tej");  
bevasarloLista.Add("Tojás");  
  
Console.WriteLine("Bevásárló lista:");  
for (int i = 0; i < bevasarloLista.Count; i++)  
{  
    Console.WriteLine($"{i + 1}. {bevasarloLista[i]}");  
}  
  
// Feladat kezelő  
List<string> feladatok = new List<string>  
{  
    "Email-ek megválaszolása",  
    "Jelentés elkészítése",  
    "Meeting 14:00"  
};  
  
// Feladat teljesítése  
feladatok.Remove("Email-ek megválaszolása");  
Console.WriteLine($"Hátralévő feladatok: {feladatok.Count}");
```

```
// Pontszámok kezelése
List<int> pontszamok = new List<int> { 85, 92, 78, 95, 88 };

// Átlag
double atlag = 0;
foreach (int pont in pontszamok)
{
    atlag += pont;
}
atalag /= pontszamok.Count;

Console.WriteLine($"Átlag: {atalag:F2}");

// Maximum
int max = pontszamok[0];
foreach (int pont in pontszamok)
{
    if (pont > max)
        max = pont;
}
Console.WriteLine($"Maximum: {max}");
```

Dictionary<TKey, TValue> - Kulcs-érték párok

A **Dictionary** kulcs-érték párokat tárol. Minden kulcs **egyedi** és egy értékhez kapcsolódik. Gyors keresést tesz lehetővé kulcs alapján.

Dictionary létrehozása

```
// Üres dictionary
Dictionary<string, int> korok = new Dictionary<string, int>();

// Inicializálás értékekkel
Dictionary<string, int> korok2 = new Dictionary<string, int>
{
    { "Anna", 25 },
    { "Béla", 30 },
    { "Cecil", 28 }
};

// Új szintaxis (C# 6.0+)
Dictionary<string, string> telefonszamok = new Dictionary<string, string>
{
    [ "Anna" ] = "06301234567",
    [ "Béla" ] = "06307654321",
    [ "Cecil" ] = "06309876543"
};

// Különböző típusok
```

```
Dictionary<int, string> azonositok = new Dictionary<int, string>();  
Dictionary<string, double> arak = new Dictionary<string, double>();  
Dictionary<string, bool> beallitasok = new Dictionary<string, bool>();
```

Elemek hozzáadása

```
Dictionary<string, int> korok = new Dictionary<string, int>();  
  
// Add metódus  
korok.Add("Anna", 25);  
korok.Add("Béla", 30);  
korok.Add("Cecil", 28);  
  
// Indexelő használata  
korok["Dóra"] = 22;  
korok["Erik"] = 35;  
  
// FIGYELEM: Már létező kulcs esetén  
// korok.Add("Anna", 26); // ArgumentException! Már létezik "Anna" kulcs  
  
// Indexelővel felülírható  
korok["Anna"] = 26; // OK, felülírja az értéket  
  
// TryAdd - csak akkor ad hozzá, ha még nincs (C# 7.0+)  
bool sikeres = korok.TryAdd("Anna", 27); // false, már létezik  
bool sikeres2 = korok.TryAdd("Ferenc", 40); // true, hozzáadva  
  
Console.WriteLine($"Elemek száma: {korok.Count}");
```

Elemek elérése

```
Dictionary<string, int> korok = new Dictionary<string, int>  
{  
    { "Anna", 25 },  
    { "Béla", 30 },  
    { "Cecil", 28 }  
};  
  
// Indexelő használata  
int annaKora = korok["Anna"]; // 25  
Console.WriteLine($"Anna kora: {annaKora}");  
  
// FIGYELEM: Nem létező kulcs esetén  
// int hiba = korok["David"]; // KeyNotFoundException!  
  
// TryGetValue - biztonságos lekérés  
if (korok.TryGetValue("Anna", out int kor))  
{  
    Console.WriteLine($"Anna {kor} éves.");  
}
```



```

    }
    else
    {
        Console.WriteLine("Anna nem található.");
    }

    // Nem létező kulcs ellenőrzés
    if (korok.TryGetValue("David", out int davidKor))
    {
        Console.WriteLine($"David {davidKor} éves.");
    }
    else
    {
        Console.WriteLine("David nincs a rendszerben.");
    }

    // Elem módosítása
    korok["Anna"] = 26;
    Console.WriteLine($"Anna új kora: {korok["Anna"]}");

```

Kulcsok és értékek

```

Dictionary<string, int> korok = new Dictionary<string, int>
{
    { "Anna", 25 },
    { "Béla", 30 },
    { "Cecil", 28 }
};

// ContainsKey - van-e ilyen kulcs
bool vanAnna = korok.ContainsKey("Anna");    // true
bool vanDavid = korok.ContainsKey("David");    // false

// ContainsValue - van-e ilyen érték
bool van25 = korok.ContainsValue(25);        // true
bool van100 = korok.ContainsValue(100);        // false

// Keys - összes kulcs
Dictionary<string, int>.KeyCollection kulcsok = korok.Keys;
Console.WriteLine("Nevek:");
foreach (string nev in kulcsok)
{
    Console.WriteLine(nev);
}

// Values - összes érték
Dictionary<string, int>.ValueCollection ertekek = korok.Values;
Console.WriteLine("Korok:");
foreach (int kor in ertekek)
{

```

```
    Console.WriteLine(kor);  
}
```

Elemek eltávolítása

```
Dictionary<string, int> korok = new Dictionary<string, int>  
{  
    { "Anna", 25 },  
    { "Béla", 30 },  
    { "Cecil", 28 },  
    { "Dóra", 22 }  
};  
  
// Remove - kulcs alapján törlés  
bool torolve = korok.Remove("Béla"); // true  
bool nemTorolve = korok.Remove("Erik"); // false (nincs ilyen kulcs)  
  
Console.WriteLine($"Elemek száma: {korok.Count}"); // 3  
  
// Clear - minden elem törlése  
korok.Clear();  
Console.WriteLine($"Elemek száma: {korok.Count}"); // 0
```

Dictionary bejárása

```
Dictionary<string, int> korok = new Dictionary<string, int>  
{  
    { "Anna", 25 },  
    { "Béla", 30 },  
    { "Cecil", 28 }  
};  
  
// foreach - kulcs-érték párok  
foreach (KeyValuePair<string, int> par in korok)  
{  
    Console.WriteLine($"{par.Key}: {par.Value} év");  
}  
  
// var kulcsszóval (rövidebb)  
foreach (var par in korok)  
{  
    Console.WriteLine($"{par.Key}: {par.Value} év");  
}  
  
// Csak kulcsok  
foreach (string nev in korok.Keys)  
{  
    Console.WriteLine(nev);  
}
```

```
// Csak értékek
foreach (int kor in korok.Values)
{
    Console.WriteLine(kor);
}

// Dekonstrukció (C# 7.0+)
foreach (var (nev, kor) in korok)
{
    Console.WriteLine($"{nev}: {kor} év");
}
```

Gyakorlati Dictionary példák

```
// Telefonkönyv
Dictionary<string, string> telefonkonyv = new Dictionary<string, string>
{
    ["Anna"] = "06301234567",
    ["Béla"] = "06307654321",
    ["Cecil"] = "06309876543"
};

Console.Write("Kinek a számát keresed? ");
string keresettNev = Console.ReadLine();

if (telefonkonyv.TryGetValue(keresettNev, out string telefonszam))
{
    Console.WriteLine($"{keresettNev} telefonszáma: {telefonszam}");
}
else
{
    Console.WriteLine($"{keresettNev} nincs a telefonkönyvben.");
}

// Termék árak
Dictionary<string, decimal> arak = new Dictionary<string, decimal>
{
    { "Kenyér", 450 },
    { "Tej", 320 },
    { "Tojás", 890 },
    { "Sajt", 1250 }
};

Console.WriteLine("=== ÁRLISTA ===");
foreach (var termek in arak)
{
    Console.WriteLine($"{termek.Key}: {termek.Value} Ft");
}

// Kosár
```

```
List<string> kosar = new List<string> { "Kenyer", "Tej", "Tej", "Tojas" };
decimal vegosszeg = 0;

Console.WriteLine("\n=== SZÁMLA ===");
Dictionary<string, int> mennyisegek = new Dictionary<string, int>();

foreach (string termek in kosar)
{
    if (mennyisegek.ContainsKey(termek))
        mennyisegek[termek]++;
    else
        mennyisegek[termek] = 1;
}

foreach (var tetel in mennyisegek)
{
    decimal ar = arak[tetel.Key];
    decimal osszeg = ar * tetel.Value;
    vegosszeg += osszeg;
    Console.WriteLine($"{tetel.Key} x{tetel.Value}: {osszeg} Ft");
}

Console.WriteLine($"Végösszeg: {vegosszeg} Ft");

// Szótár (fordítás)
Dictionary<string, string> szotar = new Dictionary<string, string>
{
    { "hello", "szia" },
    { "world", "világ" },
    { "cat", "macska" },
    { "dog", "kutya" }
};

string angol = "hello";
if (szotar.TryGetValue(angol, out string magyar))
{
    Console.WriteLine($"{angol} = {magyar}");
}

// Konfigurációs beállítások
Dictionary<string, bool> config = new Dictionary<string, bool>
{
    { "DarkMode", true },
    { "Notifications", false },
    { "AutoSave", true }
};

if (config["DarkMode"])
{
    Console.WriteLine("Sötét mód aktív");
}
```

Queue - Sor (FIFO)

A **Queue** (sor) egy **FIFO** (First In, First Out) adatszerkezet - ami először bekerül, az kerül ki először.

Queue alapok

```
// Queue létrehozása
Queue<string> sor = new Queue<string>();

// Enqueue - elem hozzáadása a sor végére
sor.Enqueue("Anna");
sor.Enqueue("Béla");
sor.Enqueue("Cecil");
// Sor: [Anna, Béla, Cecil]

// Dequeue - első elem eltávolítása és visszaadása
string elso = sor.Dequeue(); // "Anna"
// Sor most: [Béla, Cecil]

string masodik = sor.Dequeue(); // "Béla"
// Sor most: [Cecil]

// Peek - első elem lekérése (nem távolítja el)
string kovetkezo = sor.Peek(); // "Cecil"
// Sor még mindig: [Cecil]

// Count - elemek száma
Console.WriteLine($"Várókban: {sor.Count}");

// Contains - tartalmaz-e elemet
bool vanCecil = sor.Contains("Cecil"); // true

// Clear - sor ürítése
sor.Clear();
Console.WriteLine($"Várókban: {sor.Count}"); // 0
```

Queue példák

```
// Váró sor rendszer
Queue<string> varosor = new Queue<string>();

Console.WriteLine("=== VÁRÓSOR RENDSZER ===\n");

// Ügyfelek érkezése
varosor.Enqueue("Ügyfél #1");
varosor.Enqueue("Ügyfél #2");
varosor.Enqueue("Ügyfél #3");

Console.WriteLine($"Várókban: {varosor.Count} ügyfél");
```

```
// Ügyfelek kiszolgálása
while (varosor.Count > 0)
{
    string ugyfel = varosor.Dequeue();
    Console.WriteLine($"Kiszolgálva: {ugyfel}");
    Console.WriteLine($"Várakozik még: {varosor.Count}");
}

// Nyomtató sor
Queue<string> nyomtatoSor = new Queue<string>();
nyomtatoSor.Enqueue("dokument1.pdf");
nyomtatoSor.Enqueue("dokument2.docx");
nyomtatoSor.Enqueue("kep.jpg");

Console.WriteLine("\n=== NYOMTATÓ SOR ===");
while (nyomtatoSor.Count > 0)
{
    string dokumentum = nyomtatoSor.Peek();
    Console.WriteLine($"Nyomtatás: {dokumentum}");
    nyomtatoSor.Dequeue();
    System.Threading.Thread.Sleep(1000); // Szimuláljuk a nyomtatást
}

Console.WriteLine("Minden dokumentum kinyomtatva!");

// Feladat kezelő (FIFO)
Queue<string> feladatok = new Queue<string>();
feladatok.Enqueue("Email-ek");
feladatok.Enqueue("Jelentés");
feladatok.Enqueue("Meeting");

Console.WriteLine("\n=== FELADATOK (FIFO) ===");
foreach (string feladat in feladatok)
{
    Console.WriteLine($"- {feladat}");
}
```

Stack - Verem (LIFO)

A **Stack** (verem) egy **LIFO** (Last In, First Out) adatszerkezet - ami utoljára kerül be, az kerül ki először.

Stack alapok

```
// Stack létrehozása
Stack<string> verem = new Stack<string>();

// Push - elem hozzáadása a tetejére
verem.Push("Első");
verem.Push("Második");
verem.Push("Harmadik");
```

```
// Verem: [Harmadik, Második, Első]

// Pop - legfelső elem eltávolítása és visszaadása
string legfelso = verem.Pop(); // "Harmadik"
// Verem most: [Második, Első]

string kovetkezo = verem.Pop(); // "Második"
// Verem most: [Első]

// Peek - legfelső elem lekérése (nem távolítja el)
string teteje = verem.Peek(); // "Első"
// Verem még mindig: [Első]

// Count - elemek száma
Console.WriteLine($"Veremben: {verem.Count}");

// Contains - tartalmaz-e elemet
bool vanElső = verem.Contains("Első"); // true

// Clear - verem ürítése
verem.Clear();
```

Stack példák

```
// Vissza gomb (böngésző history)
Stack<string> history = new Stack<string>();

// Látogatott oldalak
history.Push("google.com");
history.Push("facebook.com");
history.Push("youtube.com");

Console.WriteLine($"Jelenlegi oldal: {history.Peek()}");

// Vissza gomb
string eloza = history.Pop();
Console.WriteLine($"Vissza: {history.Peek()}"); // facebook.com

// Szövegszerkesztő Undo funkció
Stack<string> undoStack = new Stack<string>();

string szoveg = "";
undoStack.Push(szoveg); // Üres állapot

szoveg = "Hello";
undoStack.Push(szoveg);

szoveg = "Hello World";
undoStack.Push(szoveg);

Console.WriteLine($"Jelenlegi szöveg: {szoveg}");
```

```
// Undo
undoStack.Pop(); // Eldobjuk a jelenlegi állapotot
szoveg = undoStack.Peek(); // Előző állapot
Console.WriteLine($"Undo után: {szoveg}"); // "Hello"

// Zárójelek ellenőrzése
bool EllenorizZarojelek(string szoveg)
{
    Stack<char> zarojelek = new Stack<char>();

    foreach (char c in szoveg)
    {
        if (c == '(' || c == '[' || c == '{')
        {
            zarojelek.Push(c);
        }
        else if (c == ')' || c == ']' || c == '}')
        {
            if (zarojelek.Count == 0)
                return false;

            char nyito = zarojelek.Pop();

            if ((c == ')' && nyito != '(') ||
                (c == ']' && nyito != '[') ||
                (c == '}' && nyito != '{'))
                return false;
        }
    }

    return zarojelek.Count == 0;
}

Console.WriteLine(EllenorizZarojelek("(a + b) * [c - d]")); // true
Console.WriteLine(EllenorizZarojelek("(a + b) * c")); // false
```

HashSet - Egyedi elemek halmaza

A **HashSet** **egyedi** elemeket tárol, nem engedi a duplikációt. Gyors keresést, hozzáadást és törlést biztosít.

HashSet alapok

```
// HashSet létrehozása
HashSet<int> szamok = new HashSet<int>();

// Add - elem hozzáadása
bool hozzaadva1 = szamok.Add(1); // true (új elem)
bool hozzaadva2 = szamok.Add(2); // true
bool hozzaadva3 = szamok.Add(1); // false (már benne van!)
```



```

Console.WriteLine($"Elemek: {szamok.Count}"); // 2

// Inicializálás értékekkel
HashSet<string> nevek = new HashSet<string> { "Anna", "Béla", "Cecil" };

// Contains - tartalmaz-e elemet (nagyon gyors!)
bool vanAnna = nevek.Contains("Anna"); // true

// Remove - elem törlése
bool torolve = nevek.Remove("Béla"); // true

// Clear - minden elem törlése
nevek.Clear();

```

HashSet műveletek (halmazműveletek)

```

HashSet<int> setA = new HashSet<int> { 1, 2, 3, 4, 5 };
HashSet<int> setB = new HashSet<int> { 4, 5, 6, 7, 8 };

// UnionWith - unió (összes egyedi elem)
HashSet<int> unio = new HashSet<int>(setA);
unio.UnionWith(setB);
// unio: { 1, 2, 3, 4, 5, 6, 7, 8 }

// IntersectWith - metszet (közös elemek)
HashSet<int> metszet = new HashSet<int>(setA);
metszet.IntersectWith(setB);
// metszet: { 4, 5 }

// ExceptWith - különbség (A-ban van, B-ben nincs)
HashSet<int> kulonbseg = new HashSet<int>(setA);
kulonbseg.ExceptWith(setB);
// kulonbseg: { 1, 2, 3 }

// IsSubsetOf - részhalmaz-e
bool reszHalmaz = setA.IsSubsetOf(setB); // false

// IsSupersetOf - superhalmaz-e
HashSet<int> kicsi = new HashSet<int> { 1, 2 };
bool szuperHalmaz = setA.IsSupersetOf(kicsi); // true

// Overlaps - van-e közös elem
bool vanKozos = setA.Overlaps(setB); // true

```

HashSet példák

```

// Duplikátumok eltávolítása
List<int> lista = new List<int> { 1, 2, 2, 3, 3, 3, 4, 5, 5 };

```

```
HashSet<int> egyediek = new HashSet<int>(lista);

Console.WriteLine("Eredeti lista elemei: " + string.Join(", ", lista));
Console.WriteLine("Egyedi elemek: " + string.Join(", ", egyediek));

// Látogatott oldalak (csak egyszer számít)
HashSet<string> latogatottOldalak = new HashSet<string>();

latogatottOldalak.Add("google.com");
latogatottOldalak.Add("facebook.com");
latogatottOldalak.Add("google.com"); // Nem adja hozzá újra

Console.WriteLine($"Egyedi oldalak száma: {latogatottOldalak.Count}"); // 2

// Email címek (egyediek)
HashSet<string> feliratkozok = new HashSet<string>();

bool ujFeliratkozo1 = feliratkozok.Add("anna@example.com"); // true
bool ujFeliratkozo2 = feliratkozok.Add("bela@example.com"); // true
bool ujFeliratkozo3 = feliratkozok.Add("anna@example.com"); // false
(duplikátum)

if (!ujFeliratkozo3)
{
    Console.WriteLine("Ez az email cím már fel van iratkozva!");
}

// Közös barátok
HashSet<string> annaBaratai = new HashSet<string> { "Béla", "Cecil", "Dóra" };
HashSet<string> belaBaratai = new HashSet<string> { "Anna", "Cecil", "Erik" };

HashSet<string> kozosBaratok = new HashSet<string>(annaBaratai);
kozosBaratok.IntersectWith(belaBaratai);

Console.WriteLine($"Közös barátok: {string.Join(", ", kozosBaratok)}"); // Cecil
```

Összehasonlító táblázat

Gyűjtemény	Rendezett	Egyedi	Kulcs-érték	Hozzáférés
List	Igen (index)	Nem	Nem	Index alapján
Dictionary<K,V>	Nem	Igen (kulcs)	Igen	Kulcs alapján
Queue	FIFO	Nem	Nem	Első elem
Stack	LIFO	Nem	Nem	Utolsó elem
HashSet	Nem	Igen	Nem	Elem alapján

Összetett gyakorlati példa

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        // Diák adatbázis több gyűjteménnyel

        // Dictionary: diák név → pontszám
        Dictionary<string, int> pontszamok = new Dictionary<string, int>
        {
            { "Anna", 85 },
            { "Béla", 92 },
            { "Cecil", 78 },
            { "Dóra", 95 },
            { "Erik", 88 }
        };

        // List: jelenlét (névsor)
        List<string> jelen = new List<string> { "Anna", "Cecil", "Dóra", "Erik" };

        // HashSet: beadott házi feladat
        HashSet<string> haziBeadva = new HashSet<string> { "Anna", "Béla", "Dóra"
    };

    // Jelentés
    Console.WriteLine("=== ÓRAI JELENTÉS ===\n");

    // Hiányzók
    Console.WriteLine("Hiányzók:");
    foreach (var diak in pontszamok.Keys)
    {
        if (!jelen.Contains(diak))
        {
            Console.WriteLine($"- {diak}");
        }
    }

    // Házi nem beadva
    Console.WriteLine("\nHázi nincs leadva:");
    foreach (var diak in pontszamok.Keys)
    {
        if (!haziBeadva.Contains(diak))
        {
            Console.WriteLine($"- {diak}");
        }
    }

    // Átlag számítás
    int osszeg = 0;
    foreach (var pont in pontszamok.Values)
```

```

    {
        osszeg += pont;
    }
    double atlag = (double)osszeg / pontszamok.Count;

    Console.WriteLine($"nOszttály átlag: {atlag:F2}");

    // Legjobb diákok (átlag feletti)
    Console.WriteLine("nÁtlag feletti diákok:");
    List<string> legjobbak = new List<string>();

    foreach (var diak in pontszamok)
    {
        if (diak.Value >= atlag)
        {
            legjobbak.Add(diak.Key);
            Console.WriteLine($"- {diak.Key}: {diak.Value} pont");
        }
    }

    // Rendezés pontszám szerint (csökkenő)
    Console.WriteLine("n=== RANGSOR ===");
    List<KeyValuePair<string, int>> rangsor = new List<KeyValuePair<string,
int>>(pontszamok);
    rangsor.Sort((a, b) => b.Value.CompareTo(a.Value));

    int helyezés = 1;
    foreach (var diak in rangsor)
    {
        string jelentos = jelen.Contains(diak.Key) ? "Jelen" : "Hiányzik";
        string hazi = haziBeadva.Contains(diak.Key) ? "✓" : "X";
        Console.WriteLine($"{helyezés}. {diak.Key}: {diak.Value} pont |
{jelentes} | Házi: {hazi}");
        helyezés++;
    }
}

```

Összefoglalás

List

- **Dinamikus méret:** Automatikusan növekszik
- **Indexelhető:** `list[index]`
- **Metódusok:** `Add`, `Remove`, `Contains`, `Sort`, `Reverse`
- **Használat:** Általános lista, gyakori használat

Dictionary<TKey, TValue>

- **Kulcs-érték párok:** Gyors keresés kulcs alapján
- **Egyedi kulcsok:** Minden kulcs csak egyszer

- **Metódusok:** `Add`, `Remove`, `ContainsKey`, `TryGetValue`
- **Használat:** Szótár, cache, konfigurációk

Queue

- **FIFO:** First In, First Out
- **Metódusok:** `Enqueue` (hozzáad), `Dequeue` (kivesz), `Peek`
- **Használat:** Várósort, feladat kezelés

Stack

- **LIFO:** Last In, First Out
- **Metódusok:** `Push` (hozzáad), `Pop` (kivesz), `Peek`
- **Használat:** Undo/Redo, böngésző history

HashSet

- **Egyedi elemek:** Nem engedi duplikációt
- **Gyors keresés:** $O(1)$ átlagosan
- **Halmazműveletek:** Unió, metszet, különbség
- **Használat:** Duplikátumok szűrése, halmazműveletek

Fontos:

- Mindig használd a megfelelő gyűjteményt a feladathoz!
- `using System.Collections.Generic;` - ne felejtsd el!
- Figyelj a teljesítményre nagy adatmennyiségnél

C# Alapok Tananyag - 7. Fejezet